

MATH 3260 - Chapter 10 Notes: Numerical Methods

Feedback-Revised Student Notes

Kennesaw State University - MATH 3260

Fall 2026 revision

Contents

Chapter overview	2
Section 10.1 - Gaussian Elimination with Partial Pivoting	2
Learning objectives	2
Floating-point representation	2
Error measures	2
Catastrophic cancellation	3
Detailed original example: a poor first pivot	3
Partial pivoting	3
Residual, forward error, and backward error	4
Conditioning and a perturbation bound	4
Section 10.1 terminology	5
Section 10.2 - Iterative Methods for Linear Systems	6
Stationary iterations	6
Error iteration and the spectral-radius criterion	6
Detailed example	7
Strict diagonal dominance	7
Reordering a divergent system	9
Stopping rules	9
Section 10.3 - Power Method for Dominant Eigenvalues	10
Derivation of the power method	10
Scaled iteration	10
Detailed example	10
Rayleigh quotient	11
Failure and slow-convergence cases	11
Section 10.4 - Applications of Numerical Methods	12
Quadratic least-squares regression	12
Original hiking-elevation example	12
Probability as a harmonic linear system	12
Leslie matrix and a stable age distribution	13
Model limitations	13

Chapter synthesis 14

Choosing a numerical method	14
---------------------------------------	----

These notes are synchronized with the feedback-revised student notebook. They emphasize definitions, theorems, worked mathematics, conceptual connections, and interpretation while omitting notebook-specific code and worksheet instructions.

Chapter overview

1. **10.1:** floating-point form, significant digits, absolute and relative error, Gaussian elimination, partial pivoting, conditioning, and residuals;
2. **10.2:** Jacobi and Gauss-Seidel iterations, matrix splittings, convergence, diagonal dominance, stopping rules, and divergence;
3. **10.3:** dominant eigenpairs, scaled power iteration, Rayleigh quotients, convergence rate, and failure modes;
4. **10.4:** quadratic regression with pivoting, Gauss-Seidel probability models, and Leslie population models with the power method.

Section 10.1 - Gaussian Elimination with Partial Pivoting

Learning objectives

- write decimal numbers in normalized floating-point form;
- distinguish truncation, rounding, absolute error, and relative error;
- explain why elementary arithmetic can amplify stored-value error;
- perform Gaussian elimination with simulated finite precision;
- implement partial pivoting and justify its pivot rule;
- distinguish a small residual from a small solution error;
- use the condition number to quantify sensitivity.

Floating-point representation

A base-ten normalized floating-point number has the form

$$\pm M \times 10^k, \quad 0.1 \leq M < 1.$$

For example,

$$-4382.7 = -0.43827 \times 10^4.$$

A machine with t decimal significant digits stores only t digits in the mantissa. The stored number is written $\text{fl}(x)$.

Round-half-to-even: when the discarded part is exactly half a unit in the last retained place, choose the retained digit to be even. This avoids a systematic upward bias in long sequences of ties.

Error measures

If x is exact and $\hat{x}$ is computed, then

$$E_{\text{abs}} = |x - \hat{x}|,$$

and, for $x \neq 0$,

$$E_{\text{rel}} = \frac{|x - \hat{x}|}{|x|}.$$

Relative error compares the error with the scale of the quantity. An absolute error of 0.01 is tiny for a distance of 1000 miles but large for a clearance of 0.02 inch.

First-order floating-point model

For a correctly rounded operation and no overflow or underflow, a useful model is

$$\text{fl}(x \circ y) = (x \circ y)(1 + \delta), \quad |\delta| \leq u,$$

where u is the unit roundoff. This does not say all errors are harmless: many small local errors can be magnified by subtraction, large multipliers, and ill-conditioning.

Catastrophic cancellation

Suppose x and y are close and already contain small storage errors. In

$$x - y,$$

leading digits cancel. The absolute error may stay small, but the true difference is also small, so its relative error can become very large. Elimination can create this situation when a poor pivot produces huge multipliers.

For example,

$$1.234567 - 1.234566 = 0.000001.$$

The leading digits cancel. Even if the stored inputs are accurate to several digits, the tiny difference may retain very little meaningful relative accuracy.

Detailed original example: a poor first pivot

Consider the system

$$\begin{aligned} 0.002x_1 + 2x_2 - 2x_3 &= 6.002, \\ 3x_1 - 4x_2 - 4x_3 &= -1, \\ x_1 + 4x_2 - 4x_3 &= 13. \end{aligned}$$

The exact solution is

$$\mathbf{x} = (1, 2, -1)^T.$$

Without pivoting, the first multipliers are

$$m_{21} = \frac{3}{0.002} = 1500, \quad m_{31} = \frac{1}{0.002} = 500.$$

These large multipliers magnify every error in the first row. With four-significant-digit rounding after every operation, naive elimination returns approximately

$$\hat{\mathbf{x}} = (2.000, 2.199, -0.800)^T.$$

Partial pivoting

At elimination stage k , partial pivoting chooses

$$p = \arg \max_{i \geq k} |a_{ik}|$$

and swaps rows k and p .

Why this reduces multiplier growth

After the swap, the pivot has largest magnitude in its active column. Therefore, for every row below it,

$$|m_{ik}| = \left| \frac{a_{ik}}{a_{kk}} \right| \leq 1.$$

Partial pivoting does not eliminate rounding error, but it prevents the immediate creation of multipliers such as 1500 in the example.

The fix to the small-pivot problem is to choose a better pivot before eliminating. Partial pivoting implements that idea systematically.

Residual, forward error, and backward error

For a computed solution $\hat{\mathbf{x}}$ of $A\mathbf{x} = \mathbf{b}$, the residual is

$$\mathbf{r} = \mathbf{b} - A\hat{\mathbf{x}}.$$

- **Forward error:** $\mathbf{x} - \hat{\mathbf{x}}$; it requires knowledge of the exact solution.
- **Residual:** measures how well the computed vector satisfies the original equations.
- **Backward error:** asks how much A or $\mathbf{b}$ must change to make $\hat{\mathbf{x}}$ exact.

A small residual does not by itself guarantee a small forward error if the system is ill-conditioned.

Conditioning and a perturbation bound

Definition: condition number

For an invertible matrix and a consistent norm,

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Perturbation bound

Suppose $A\mathbf{x} = \mathbf{b}$ and $A(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{b} + \Delta\mathbf{b}$. Then

$$A\Delta\mathbf{x} = \Delta\mathbf{b}, \quad \Delta\mathbf{x} = A^{-1}\Delta\mathbf{b}.$$

Therefore

$$\|\Delta\mathbf{x}\| \leq \|A^{-1}\| \|\Delta\mathbf{b}\|.$$

Also $\|\mathbf{b}\| = \|A\mathbf{x}\| \leq \|A\| \|\mathbf{x}\|$, so

$$\frac{1}{\|\mathbf{x}\|} \leq \frac{\|A\|}{\|\mathbf{b}\|}.$$

Multiplying the inequalities gives

$$\frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \frac{\|\Delta \mathbf{b}\|}{\|\mathbf{b}\|}.$$

Thus the condition number is an error-amplification factor in the worst case.

Section 10.1 terminology

Term	Meaning
Significant digits	Retained meaningful digits in a stored number
Unit roundoff	Bound in the relative floating-point model
Cancellation	Loss of relative accuracy in subtracting nearby numbers
Partial pivoting	Largest-magnitude pivot selection in the active column
Residual	$\mathbf{b} - A\hat{\mathbf{x}}$
Forward error	Difference between exact and computed solutions
Condition number	Worst-case sensitivity factor $\ A\ \ A^{-1}\ $
Ill-conditioned	Small input changes can cause large solution changes

Section 10.2 - Iterative Methods for Linear Systems

Stationary iterations

Write

$$A = D + L + U,$$

where D is diagonal, L strictly lower triangular, and U strictly upper triangular.

Jacobi

$$D\mathbf{x}^{(k+1)} = \mathbf{b} - (L + U)\mathbf{x}^{(k)},$$

so

$$\mathbf{x}^{(k+1)} = B_J\mathbf{x}^{(k)} + \mathbf{c}_J, \quad B_J = -D^{-1}(L + U).$$

Gauss-Seidel

$$(D + L)\mathbf{x}^{(k+1)} = \mathbf{b} - U\mathbf{x}^{(k)},$$

so

$$\mathbf{x}^{(k+1)} = B_{GS}\mathbf{x}^{(k)} + \mathbf{c}_{GS}, \quad B_{GS} = -(D + L)^{-1}U.$$

Gauss-Seidel immediately uses the newest available components.

Jacobi uses only values from the previous iterate. Gauss-Seidel immediately reuses the newest values, so improvements made early in an iteration influence later components immediately. This is why Gauss-Seidel often reduces the error faster.

Why iterative methods?

Direct methods such as Gaussian elimination reach a solution through a finite algebraic process. Iterative methods instead generate approximations

$$\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$$

that aim to converge to the solution. For very large sparse systems, this can save both memory and computation. The tradeoff is that convergence must be analyzed.

Error iteration and the spectral-radius criterion

Let $\mathbf{x}^*$ be the exact solution and let $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$. Since the exact solution is a fixed point,

$$\mathbf{e}^{(k+1)} = B\mathbf{e}^{(k)} = B^{k+1}\mathbf{e}^{(0)}.$$

Therefore the method converges for every initial vector precisely when

$$B^k \rightarrow 0.$$

For a finite-dimensional matrix,

$$\boxed{B^k \rightarrow 0 \iff \rho(B) < 1},$$

where $\rho(B)$ is the maximum eigenvalue magnitude.

Proof idea in the diagonalizable case

If $B = P\Lambda P^{-1}$, then

$$B^k = P\Lambda^k P^{-1}.$$

If every $|\lambda_i| < 1$, then $\Lambda^k \rightarrow 0$. If some $|\lambda_i| \geq 1$, an eigenvector in that direction does not decay. The general Jordan-form proof reaches the same conclusion because polynomial factors cannot overcome exponential decay when $|\lambda| < 1$.

Detailed example

Solve iteratively

$$\begin{aligned} 10x_1 - x_2 + 2x_3 &= 6, \\ -x_1 + 11x_2 - x_3 &= 22, \\ 2x_1 - x_2 + 10x_3 &= -10. \end{aligned}$$

The exact solution is $(1, 2, -1)^T$. The update equations are

$$\begin{aligned} x_1 &= \frac{6 + x_2 - 2x_3}{10}, \\ x_2 &= \frac{22 + x_1 + x_3}{11}, \\ x_3 &= \frac{-10 - 2x_1 + x_2}{10}. \end{aligned}$$

From matrix form to component form

A row

$$a_{ii}x_i + \sum_{j \neq i} a_{ij}x_j = b_i$$

can be solved for its diagonal variable:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij}x_j \right).$$

Doing this row by row produces the component update formulas.

Strict diagonal dominance

A is strictly diagonally dominant by rows when

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for every } i.$$

Full Jacobi convergence proof

Define

$$q = \max_i \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} < 1.$$

From the component error equation,

$$e_i^{(k+1)} = -\frac{1}{a_{ii}} \sum_{j \neq i} a_{ij} e_j^{(k)}.$$

Hence

$$|e_i^{(k+1)}| \leq \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} \|\mathbf{e}^{(k)}\|_\infty \leq q \|\mathbf{e}^{(k)}\|_\infty.$$

Taking the maximum over i gives

$$\|\mathbf{e}^{(k+1)}\|_\infty \leq q \|\mathbf{e}^{(k)}\|_\infty,$$

and induction yields

$$\|\mathbf{e}^{(k)}\|_\infty \leq q^k \|\mathbf{e}^{(0)}\|_\infty \rightarrow 0.$$

Full Gauss-Seidel convergence proof

For the i th updated component,

$$e_i^{(k+1)} = -\frac{1}{a_{ii}} \left(\sum_{j < i} a_{ij} e_j^{(k+1)} + \sum_{j > i} a_{ij} e_j^{(k)} \right).$$

Induct on i . Assume the already updated components satisfy $|e_j^{(k+1)}| \leq q \|\mathbf{e}^{(k)}\|_\infty$. Since $q < 1$,

$$\begin{aligned} |e_i^{(k+1)}| &\leq \frac{q \sum_{j < i} |a_{ij}| + \sum_{j > i} |a_{ij}|}{|a_{ii}|} \|\mathbf{e}^{(k)}\|_\infty \\ &\leq \frac{\sum_{j \neq i} |a_{ij}|}{|a_{ii}|} \|\mathbf{e}^{(k)}\|_\infty \\ &\leq q \|\mathbf{e}^{(k)}\|_\infty. \end{aligned}$$

Thus Gauss-Seidel is also a contraction.

Reordering a divergent system

For

$$x + 4y = 5, \quad 3x + y = 4,$$

Jacobi in the displayed order uses

$$x^{(k+1)} = 5 - 4y^{(k)}, \quad y^{(k+1)} = 4 - 3x^{(k)},$$

and diverges from $(0, 0)$. Reorder the equations:

$$3x + y = 4, \quad x + 4y = 5.$$

The new coefficient matrix is strictly diagonally dominant, and both methods converge to $(1, 1)$.

Stopping rules

A practical method may stop when

$$\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_{\infty} \leq \tau_{\text{abs}} + \tau_{\text{rel}} \|\mathbf{x}^{(k+1)}\|_{\infty},$$

and should also check

$$\|\mathbf{b} - A\mathbf{x}^{(k+1)}\|_{\infty}.$$

The change between iterates detects stagnation; the residual tests the equations. A maximum iteration count prevents an infinite loop.

Direct versus iterative methods

Situation	Typical preference
Small/medium dense system	direct factorization
Many right-hand sides, same A	LU factorization
Very large sparse system	iterative method
Need a solve without convergence testing	direct method
Favorable spectral/diagonal structure	iterative method can be very efficient

Section 10.3 - Power Method for Dominant Eigenvalues

An eigenvalue λ_1 is dominant when

$$|\lambda_1| > |\lambda_j| \quad (j \geq 2).$$

The corresponding eigenspace contains dominant eigenvectors.

Derivation of the power method

Assume A is diagonalizable with eigenbasis $\mathbf{v}_1, \dots, \mathbf{v}_n$, and write

$$\mathbf{x}^{(0)} = c_1 \mathbf{v}_1 + \dots + c_n \mathbf{v}_n, \quad c_1 \neq 0.$$

Then

$$A^k \mathbf{x}^{(0)} = c_1 \lambda_1^k \mathbf{v}_1 + \sum_{j=2}^n c_j \lambda_j^k \mathbf{v}_j.$$

Factor out λ_1^k :

$$A^k \mathbf{x}^{(0)} = \lambda_1^k \left[c_1 \mathbf{v}_1 + \sum_{j=2}^n c_j \left(\frac{\lambda_j}{\lambda_1} \right)^k \mathbf{v}_j \right].$$

Every ratio tends to zero. After scaling, the direction approaches $\mathbf{v}_1$. The leading asymptotic rate is approximately

$$\left| \frac{\lambda_2}{\lambda_1} \right|^k.$$

Goal of the power method

The power method estimates the **dominant eigenvalue** and a corresponding eigenvector without computing every eigenpair. It is especially useful when the matrix is large and the dominant long-run mode is the main quantity of interest.

Scaled iteration

To avoid overflow or underflow, compute

$$\mathbf{y}^{(k+1)} = A \mathbf{x}^{(k)}, \quad \alpha_{k+1} = \|\mathbf{y}^{(k+1)}\|_{\infty}, \quad \mathbf{x}^{(k+1)} = \frac{\mathbf{y}^{(k+1)}}{\alpha_{k+1}}.$$

Scaling changes magnitude but not direction.

Detailed example

Use

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix}, \quad \mathbf{x}^{(0)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

The eigenvalues are 5 and 2. The dominant eigenvector is a multiple of $(1, 1)^T$.

Rayleigh quotient

For nonzero $\mathbf{x}$,

$$R_A(\mathbf{x}) = \frac{\mathbf{x}^T A \mathbf{x}}{\mathbf{x}^T \mathbf{x}}.$$

If $A\mathbf{x} = \lambda\mathbf{x}$, then

$$R_A(\mathbf{x}) = \frac{\mathbf{x}^T (\lambda\mathbf{x})}{\mathbf{x}^T \mathbf{x}} = \lambda.$$

For an approximate eigenvector, the quotient gives an eigenvalue estimate. For symmetric A it has especially strong accuracy properties and lies between the smallest and largest eigenvalues.

The normalization scale factor and the Rayleigh quotient are two eigenvalue estimates. The scale factor is cheap because normalization already computes it. The Rayleigh quotient uses the whole vector and is especially effective for symmetric matrices.

Convergence-speed interpretation

The asymptotic speed is controlled mainly by

$$\left| \frac{\lambda_2}{\lambda_1} \right|.$$

A ratio near 0.1 gives rapid convergence, while a ratio near 0.99 gives slow convergence.

Failure and slow-convergence cases

1. If $c_1 = 0$, the initial vector has no dominant-eigenvector component.
2. If two eigenvalues share the largest magnitude, the direction may oscillate or remain in a multidimensional invariant subspace.
3. If $|\lambda_2/\lambda_1|$ is close to one, convergence is slow.
4. A negative dominant eigenvalue can cause sign alternation; direction convergence is interpreted up to sign.
5. A defective or strongly nonnormal matrix may show transient behavior not predicted by eigenvalue magnitude alone.

A reliable implementation monitors direction change, Rayleigh quotient change, and the eigenresidual

$$\|A\mathbf{x} - R_A(\mathbf{x})\mathbf{x}\|.$$

Section 10.4 - Applications of Numerical Methods

Quadratic least-squares regression

For data (x_i, y_i) and

$$p(x) = a_0 + a_1x + a_2x^2,$$

minimize

$$S(a_0, a_1, a_2) = \sum_{i=1}^n [y_i - (a_0 + a_1x_i + a_2x_i^2)]^2.$$

Setting all three partial derivatives to zero gives

$$\begin{bmatrix} n & \sum x_i & \sum x_i^2 \\ \sum x_i & \sum x_i^2 & \sum x_i^3 \\ \sum x_i^2 & \sum x_i^3 & \sum x_i^4 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \sum x_i^2 y_i \end{bmatrix}.$$

These are the normal equations $X^T X \mathbf{a} = X^T \mathbf{y}$. Partial pivoting is appropriate when solving them, although QR factorization is generally more stable for serious least-squares computation.

Original hiking-elevation example

A fictional trail records elevation gain after hours $x = 0, 1, \dots, 5$:

$$120, 182, 255, 329, 388, 415.$$

The needed sums are

$$\sum x = 15, \quad \sum x^2 = 55, \quad \sum x^3 = 225, \quad \sum x^4 = 979,$$

and

$$\sum y = 1689, \quad \sum xy = 5306, \quad \sum x^2 y = 20746.$$

Thus

$$\begin{bmatrix} 6 & 15 & 55 \\ 15 & 55 & 225 \\ 55 & 225 & 979 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 1689 \\ 5306 \\ 20746 \end{bmatrix}.$$

Probability as a harmonic linear system

At an interior state, the probability of eventual success equals the weighted average of the success probabilities at the next states. Consider four trail intersections with success probabilities $p_1, \dots, p_4$:

$$p_1 = \frac{1}{2}p_2,$$

$$p_2 = \frac{1}{3}(p_1 + p_3 + p_4),$$

$$p_3 = \frac{1}{2}(p_2 + 1),$$

$$p_4 = \frac{1}{3}(p_2 + 1 + 0).$$

The constants 1 and 0 represent immediate success and failure exits. The exact solution is

$$(p_1, p_2, p_3, p_4) = \left(\frac{1}{4}, \frac{1}{2}, \frac{3}{4}, \frac{1}{2} \right).$$

Gauss-Seidel is natural because each newly updated probability is used immediately.

Leslie matrix and a stable age distribution

For age classes, a Leslie matrix has fertility coefficients in the first row and survival probabilities on the subdiagonal. Use the fictional trail-songbird matrix

$$L = \begin{bmatrix} 0 & 1.1 & 1.6 & 0.5 \\ 0.72 & 0 & 0 & 0 \\ 0 & 0.65 & 0 & 0 \\ 0 & 0 & 0.55 & 0 \end{bmatrix}.$$

If $\mathbf{n}_k$ is the age-count vector, then

$$\mathbf{n}_{k+1} = L\mathbf{n}_k.$$

A stable age distribution has constant proportions, so

$$L\mathbf{v} = \lambda\mathbf{v}.$$

The dominant eigenvalue is the long-run per-period growth factor; a normalized positive dominant eigenvector gives the stable proportions.

Model limitations

Numerical output is conditional on the model:

- regression extrapolation can be unreliable outside the data interval;
- maze probabilities assume fixed transition probabilities and eventual absorption;
- Leslie models assume time-invariant fertility and survival and ignore density dependence;
- convergence of an algorithm does not prove that the underlying scientific assumptions are correct.

A responsible analysis reports algorithmic error, data uncertainty, and model limitations separately.

Chapter synthesis

Numerical linear algebra studies how mathematical formulas behave when implemented with finite storage and finite computation.

1. Partial pivoting controls elimination multipliers and usually improves stability.
2. Conditioning belongs to the problem; stability belongs to the algorithm.
3. Jacobi and Gauss-Seidel are fixed-point iterations whose errors evolve by matrix powers.
4. Strict diagonal dominance gives a concrete contraction proof.
5. The power method isolates the dominant eigendirection at a rate governed by an eigenvalue ratio.
6. Regression, probability, and population models reuse the same matrix algorithms in different applications.
7. Residuals, stopping rules, and model assumptions must be checked—not merely the final number.

Choosing a numerical method

The question is not only “Can this method solve the system?” but also “Why choose this method here?” Direct methods are attractive for moderate dense problems and repeated solves after factorization. Iterative methods become attractive for very large sparse problems. Pivoting improves numerical stability, while conditioning tells us how sensitive the mathematical problem itself is to perturbations. For eigenvalue problems, the power method is useful when only the dominant mode is needed rather than the full spectrum.